



2026_포트폴리오

박지원

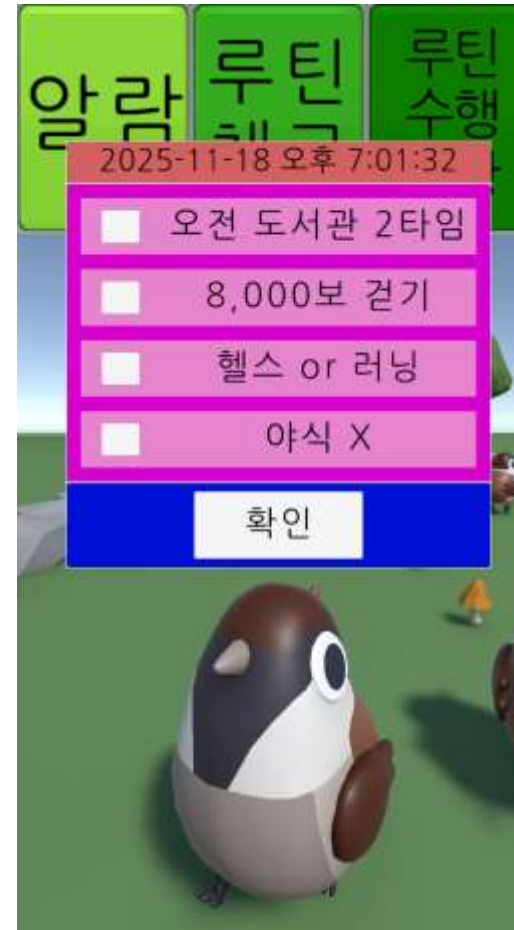


목차

1. 개요
2. View
3. Presenter
4. Model
5. Manager
6. GameStartManager
7. 후기

1. 개요

- 퇴사 후, 진행한 1인 프로젝트 입니다.



- 프로젝트 전체 github 주소입니다.
- <https://github.com/pjw960316/HabitGame>

2. View

- View는 유저에게 보여지는 부분을 담당합니다. (UI, FieldObject)
- View는 멍청하기 때문에 Presenter와 Model을 들고 있지 않고, 인풋에 반응해서 Presenter에게 알리기만 합니다. 이것을 UniRx로 구현했습니다.

```
public abstract class FieldObjectAnimalBase : FieldObjectBase
```

```
private readonly Subject<Collision> _onCollision = new();
```

```
#endregion
```

```
#region 2. Properties
```

```
1 usage 2 JIWON PARK
```

```
public IObservable<Collision> OnCollision => _onCollision;
```

```
Event-function 2 JIWON PARK
```

```
private void OnCollisionEnter(Collision other)
```

```
{
```

```
    _currentCollision = other;
```

```
    _onCollision.OnNext(other);
```

```
}
```

- View가 필드로 Presenter를 들고 있지 않습니다.
- 그러므로 IObservable을 public으로 제공하여 Presenter가 이벤트를 처리할 수 있도록 합니다.

2. View

- View가 Model을 들고 있지 않기 때문에 Data를 변경하지 않게 됩니다.
- 전 직장에서 View에서 data를 조작하도록 구현한 적이 있었습니다.
그 결과 공용으로 사용되던 데이터가 어디서 변경되었는지 추적이 되지 않았고,
View가 Close 되면 Data에서 null이 참조되는 문제도 발생했습니다.
- 유저 인풋으로 데이터가 변경이 되어야 해도, 그 사실을 View는 알지 못합니다.
View는 그저 인풋에 대해 반응만 할 뿐이며, 모든 판단은 Presenter가 해야 합니다.

```
0+1 usages  JIWON PARK
protected sealed override void BindEvent()
{
    _confirmButton.OnClick.AddListener(OnClickConfirmButton);
}

1 usage  JIWON PARK
private void OnClickConfirmButton()
{
    _onConfirmed.OnNext(default);
}
```

View는 버튼이 눌리면 Presenter에게 알리기만 합니다.

2. View

- 로직이 중요하지 않은 단순 Transform 계산이나 Render 수정의 경우, View에서 처리해도 무방하다고 판단했습니다.

```
public abstract class FieldObjectAnimalBase : FieldObjectBase
```

```
    Frequently called 1 usage JIWON PARK
    public void ChangeAnimation(int enumKey)
    {
        _animalAnimator.SetInteger(_animalAnimatorIntegerParameter, enumKey);
    }

    Frequently called 3 usages JIWON PARK
    public void ChangeAnimalPath(int angle)
    {
        FieldObjectTransform.Rotate(new Vector3(0, angle, 0));
        UpdateAnimalMovement();
    }

    Frequently called 2 usages JIWON PARK
    public void RotateToFaceCollisionObject()
    {
        var path = _currentCollision.transform.position - FieldObjectTransform.position;
        var facePath = Quaternion.LookRotation(path);

        FieldObjectTransform.rotation = facePath;
    }

    Frequently called 2 usages JIWON PARK
    public void ChangeAnimalSpeed(float speed)
    {
        _animalSpeed = speed;

        UpdateAnimalMovement();
    }

    Frequently called 1 usage JIWON PARK
    public void ChangeAnimalDefaultSpeed()
    {
        ChangeAnimalSpeed(DefaultAnimalSpeed);
    }
```

- 초기에는 모든 동작을 Presenter에서 처리해야 한다고 생각했습니다.
- 하지만 GameObject의 Component 요소가 잠재적으로 게임 로직을 필요로 하지 않는 경우라면, View에서 처리하는 것이 오히려 더 깔끔하다고 판단했습니다.
- 이렇게 View에서 처리 범위를 넓히면 Presenter에 대한 의존도 역시 줄어듭니다

2. View

- Base 클래스에서 Awake() 내부의 호출 순서를 정의합니다.

```
public abstract class UIPopupBase : MonoBehaviour, IView
```

```
#region 3. Constructor
```

```
Event function ◊ JIWON PARK
```

```
private void Awake()
```

```
{
```

```
    InitializeEPopupKey();
```

```
    Initialize();
```

```
    CreatePresenterByManager();
```

```
    BindEvent();
```

```
}
```

```
5 usages 4 overrides ◊ JIWON PARK
```

```
protected virtual void Initialize()
```

```
{
```

```
    _uiToastManager = UIToastManager.Instance;
```

```
    _presenterManager = PresenterManager.Instance;
```

```
}
```

```
1 usage 4 overrides JIWON PARK
```

```
protected abstract void InitializeEPopupKey();
```

```
1 usage 4 overrides JIWON PARK
```

```
protected abstract void CreatePresenterByManager();
```

```
1 usage 4 overrides JIWON PARK
```

```
protected abstract void BindEvent();
```

- 하위 타입을 구현할 때는 호출 순서를 고민할 필요가 없습니다.
- Virtual을 사용해 하위 타입에서도 공통 동작이 호출되도록 보장합니다.
- Abstract를 사용해 하위 타입에 필요한 책임을 부여합니다.

3. Presenter

- Presenter는 View와 Model 사이를 연결하는 중재자 역할을 합니다.

상속 구조 : AlarmPresenter -> UIPresenterBase -> PresenterBase -> IPresenter

```
public abstract class PresenterBase : IPresenter
```

```
protected IView _view;  
protected IModel _model;
```

```
3 usages 9 overrides JIWON PARK  
public virtual void Initialize(IView view)  
{  
    _soundManager = SoundManager.Instance;  
    _uiToastManager = UIToastManager.Instance;  
    _uiManager = UIManager.Instance;  
    _myCharacterManager = MyCharacterManager.Instance;  
    _scriptableObjectManager = ScriptableObjectManager.Instance;  
    _stringManager = StringManager.Instance;  
    _presenterManager = PresenterManager.Instance;  
    _serverManager = MockServerManager.Instance;  
    _view = view;  
  
    InitializeView();  
    InitializeModel();  
}
```

```
1 usage 9 overrides JIWON PARK  
protected abstract void InitializeView();  
  
1 usage 6 overrides JIWON PARK  
protected abstract void InitializeModel();
```

```
public class AlarmPresenter : UIPresenterBase
```

```
0+5 usages JIWON PARK  
protected sealed override void InitializeView()  
{  
    base.InitializeView();  
  
    _alarmPopup = _view as UIAlarmPopup;  
    ExceptionHelper.CheckNullException(_alarmPopup, "_alarmPopup");  
}
```

```
0+1 usages JIWON PARK  
protected sealed override void InitializeModel()  
{  
    _alarmData = _scriptableObjectManager.GetScriptableObject<AlarmData>();  
    ExceptionHelper.CheckNullException(_alarmData, "_alarmData");  
}
```

- View와 Model을 인터페이스로 Base 클래스에서 구현합니다.
- Presenter의 Initialize() 메서드는 View를 인자로 받아 실행됩니다. 그 이유는 다음 장에서 설명드리겠습니다.
- 하위 타입에서는 구체적인 View와 Model로 초기화합니다.

3. Presenter

- Presenter는 View 와 Model 사이를 연결하는 중재자 역할을 합니다.

```
public class PresenterManager : ManagerBase<PresenterManager>

// note : Factory Pattern
6 usages  JIWON PARK
public void CreatePresenter<TPresenter>(IView view) where TPresenter : PresenterBase, new()
{
    var presenter = new TPresenter();

    presenter.Initialize(view);
    presenter.SetView();
    presenter.BindEvent();

    _livedPresenterHashSet.Add(presenter);
}

Frequently called 1 usage  JIWON PARK
public void TerminatePresenter(PresenterBase presenter)
{
    if (_livedPresenterHashSet.Contains(presenter))
    {
        _livedPresenterHashSet.Remove(presenter);
    }
}
```

- View가 생성될 때
PresenterManager에게
자신을 인자로 전달하며
Presenter 생성을
요청합니다.
- PresenterManager는
Factory 역할을 하여,
View 생성 시 알맞은
Presenter를 생성합니다.

3. Presenter

- Presenter는 View와 Model의 IObservable을 구독할 책임이 있습니다.

```
public abstract class FieldObjectAnimalPresenterBase : FieldObjectPresenterBase
```

```
public override void BindEvent()
{
    base.BindEvent();

    _fieldObjectAnimal.OnCollision.Subscribe(OnCollision).AddTo(_disposable);
    _animalData.OnAnimalStateChanged.Subscribe(OnChangeAnimalState).AddTo(_disposable);

    // 걷기 방향은 모든 동물에서 동작시킨다.
    BindDirectionChangeInterval();
    BindChangeRandomDirection();
}
```

```
public void OnChangeAnimalState(EAnimalState changedState)
{
    _currentAnimalState = changedState;
    _fieldObjectAnimal.ChangeAnimation((int)_currentAnimalState);

    // Log
    //Debug.Log($"{_fieldObjectAnimal.name}의 현재 상태 : {_currentAnimalState}");

    if (_currentAnimalState == EAnimalState.WALK)
    {
        _fieldObjectAnimal.ChangeAnimalDefaultSpeed();
    }
}
```

- View의 IObservable을 Subscribe 하여 이벤트를 핸들링합니다.
- Model의 IObservable도 처리하여 상태를 관리하는 주체가 됩니다.
- 복잡한 핸들링 로직 역시 Presenter에서 모아 깔끔하게 처리할 수 있습니다.

3. Presenter

- Presenter는 View와 일대일로 대응되는 것이 좋다고 판단했습니다.

Derived types of 'UIPresenterBase'	
AlarmPresenter	Assembly-CSharp
AlarmTimerPresenter	Assembly-CSharp
RoutineCheckPresenter	Assembly-CSharp
RoutineRecordPresenter	Assembly-CSharp

Derived types of 'UIPopupBase'	
UIAlarmPopup	Assembly-CSharp
UIAlarmTimerPopup	Assembly-CSharp
UIRoutineCheckPopup	Assembly-CSharp
UIRoutineRecordPopup	Assembly-CSharp

```
public class AlarmPresenter : UIPresenterBase
{
    #region 1. Fields

    private UIAlarmPopup _alarmPopup;
    private AlarmData _alarmData;
}
```

```
public class AlarmTimerPresenter : UIPresenterBase
{
    #region 1. Fields

    private UIAlarmTimerPopup _alarmTimerPopup;
    private AlarmData _alarmData;
}
```

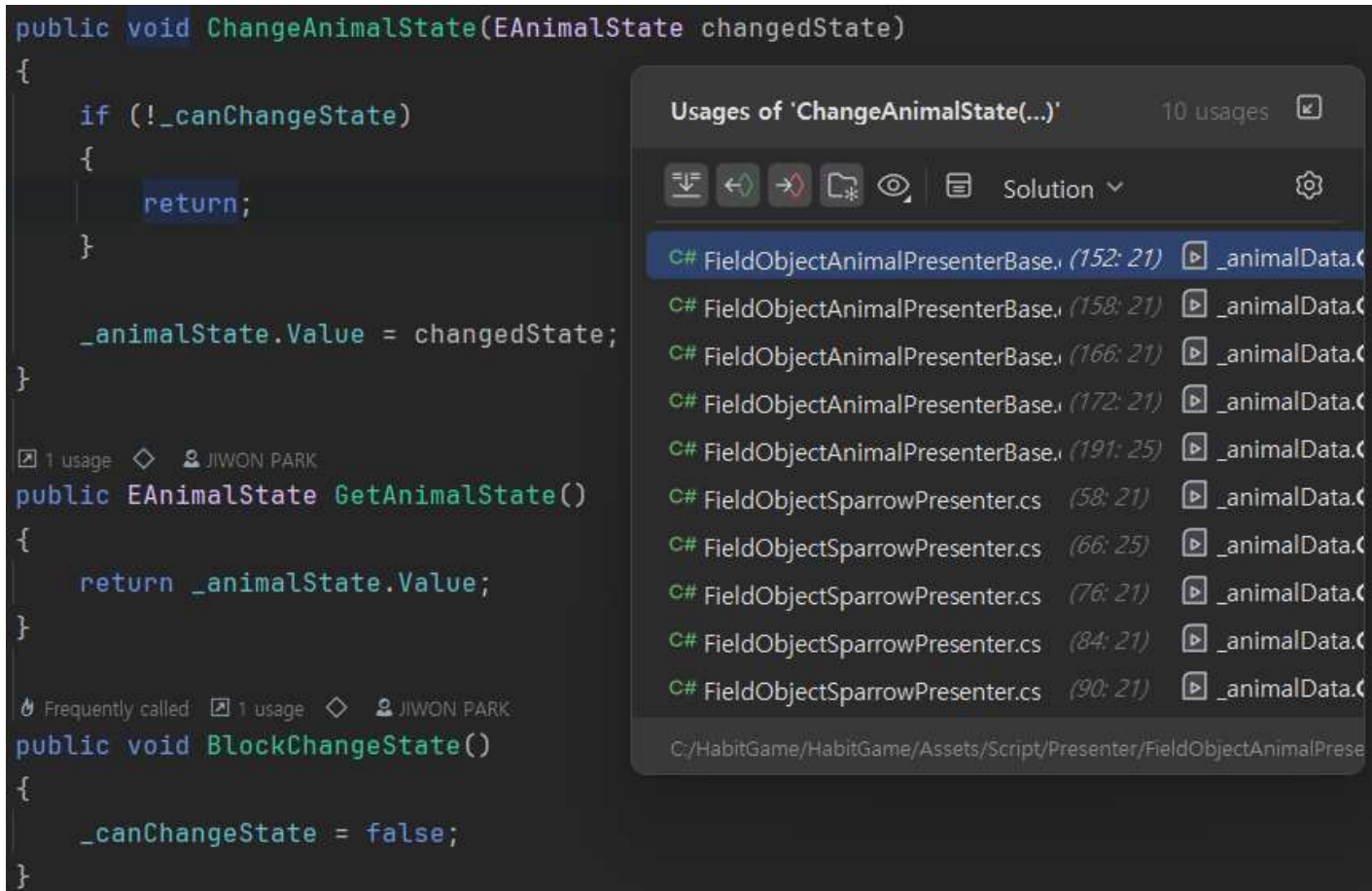
```
protected sealed override void InitializeModel()
{
    _alarmData = _scriptableObjectManager.GetScriptableObject<AlarmData>();
    ExceptionHelper.CheckNullException(_alarmData, "_alarmData");
}
```

- 개발 초기에는 AlarmPresenter와 AlarmTimerPresenter가 동일한 Model을 사용하고 있었습니다. 그래서 두 개의 View가 하나의 Presenter를 공유하도록 구현했습니다.
- 하지만 하나의 Presenter로 두 개의 View의 생명주기를 관리하기는 어려웠고, 여러 분기 처리도 필요해졌습니다.
- 따라서 View와 Presenter를 1:1로 대응시키고, View와 Presenter가 묶인 것을 V-P라고 할 때, 여러 개의 V-P가 하나의 Model을 참조하는 구조를 채택하기로 했습니다.

(예시 : 2개의 V-P가 AlarmData Model을 참조)

4. Model

- Presenter가 데이터를 요청할 때, 이를 전달해 줄 책임이 있습니다.



- 콘텐츠와 관련된 데이터를 Model에서 관리하며, Presenter의 요청을 받아 데이터를 전달합니다.
(View는 Rx가 아니라면 호출할 수 없습니다.)
- Call-by-ref인지 call-by-value인지는 전달하는 params의 타입만 확인하면 됩니다.
이때 원본에 변경이 발생하는지 주의해야 합니다.

4. Model

- 캡슐화를 최우선적으로 지킵니다.

```
[CreateAssetMenu(fileName = "AlarmData", menuName = "ScriptableObjects/AlarmData")]  
1 asset usage 5 usages JIWON PARK  
public class AlarmData : ScriptableObject, IModel
```

```
private readonly Dictionary<EAlarmButtonType, string> _sleepingAudioClipPathDictionary = new();  
  
[SerializeField] private SerializedDictionary<EAlarmButtonType, AudioClip> _sleepingAudioClipDictionary = new();  
[SerializeField] private SerializedDictionary<EAlarmButtonType, float> _sleepingAudioPlayTimeDictionary = new();  
[SerializeField] private AudioClip _alarmAudioClip; 1 Serializable
```

```
1 usage JIWON PARK  
public AudioClip AlarmAudioClip => _alarmAudioClip;
```

```
1 Frequently called 3 usages  
public AudioClip LatestSleepingAudioClip { get; private set; }
```

```
1 Frequently called 4 usages  
public float LatestSleepingAudioPlayTime { get; private set; }
```

```
1 usage JIWON PARK  
public ImmutableDictionary<EAlarmButtonType, string> SleepingAudioClipPathDictionary =>  
    _sleepingAudioClipPathDictionary.ToImmutableDictionary();
```

```
1 usage JIWON PARK  
public ImmutableDictionary<EAlarmButtonType, float> SleepingAudioPlayTimeDictionary =>  
    _sleepingAudioPlayTimeDictionary.ToImmutableDictionary();
```

- 필드는 private으로 관리합니다.
Public property로 getter만
오픈합니다.
예외 처리 시에만 Setter를
허용합니다.
- Container의 경우 Immutable
Type을 사용해
외부에서 내부 원소가 변경되는
것을 차단합니다.

4. Model

- 클라 – 서버 구조에서는 Model이 패킷으로 서버에 요청을 보내 갱신합니다.

```
public async UniTask<EServerResult> RequestServerValidation()
{
    var serverResult = await GenerateMockServerLogic();

    if (serverResult == EServerResult.SUCCESS)
    {
        return serverResult;
    }

    return EServerResult.FAIL;
}
```

```
// note
// 실제 서버에서는 서버로직에 따라서 다양한 enum이 return 될 것
Frequently called 1 usage JIWON PARK
private async UniTask<EServerResult> GenerateMockServerLogic()
{
    await GenerateServerDelay();

    return EServerResult.SUCCESS;
}

Frequently called 1 usage JIWON PARK
private async UniTask GenerateServerDelay()
{
    await UniTask.Delay(SERVER_DELAY_MILLISECONDS);
}
```

- 클라 – 서버 구조에서는 간단한 UI가 아닌 이상, 데이터는 서버에게 패킷으로 받아옵니다. 회사에서 라이브 서버 지연으로 ERROR Enum이 반환되며 null이 떠서 고생했던 기억도 있습니다.
- Presenter가 Request Packet을 보내면, 서버 지연 후 서버 개발자가 패킷을 전달해줍니다. 클라-서버 SharedType으로 구현된 데이터 타입이 결국 모델의 구성 요소가 됩니다.
- MockServer로 지연만 흉내 냈고, 이번 개인 프로젝트에서는 ScriptableObject와 XML을 활용했습니다.

5. Manager

- Manager는 MVP의 묶음을 관리하는 책임만 갖습니다.

```
public class UIManager : ManagerBase<UIManager>
{
    public void OpenPopupByStringKey(EPopupKey popupKey, Transform transform)
    {
        var popupPrefab = _popupData.GetPopupByEPopupKey(popupKey);

        if (popupPrefab == null)
        {
            throw new InvalidOperationException("<UNK> <UNK> <UNK> <UNK> <UNK> <UNK>.");
        }

        var popup = Object.Instantiate(popupPrefab, transform).GetComponent<UIPopupBase>();

        if (popup != null)
        {
            _popupDictionary[popupKey] = popup;
            _openedPopupKeyList.Add(popupKey);

            _onOpenPopup.OnNext(popupKey);
        }
    }
}
```

- 일반적인 게임에는
UI 팝업 콘텐츠가 매우 많습니다.
- UI들을 여는 것, 닫는 것, 열린 UI의
생명주기 관리처럼
‘관리’의 영역만 담당해야 합니다.
- 각 UI가 담당해야 할 코드는
각 UI의 Presenter로 옮기면 됩니다.

5. Manager

- **Manager는 MVP의 묶음을 관리하는 책임만 있습니다.**
- 회사에서 진행하는 대형 프로젝트의 경우, 볼륨이 크기 때문에 필연적으로 Manager 클래스에 코드가 집중됩니다.
1,000줄이 넘는 Manager Class도 있었고,
가독성이 떨어지며 Manager의 책임이 지나치게 커지게 됩니다.
- 저는 지금까지 Singleton Manager에 의존하는 코딩을 해왔습니다.
Manager에 코드를 많이 작성하곤 했습니다.
그 결과, 팀원과의 의존성도 높아지고,
심지어 제가 이전에 작업한 코드에서도 변경 사항이 생기면 수정이 쉽지 않았습니다.
- SoundManager에 모든 음악 관련 메서드를 넣어두니 관리가 제대로 되지 않았습니다.
그래서 각 메서드가 관리 책임을 가지는지 판단했고,
그렇지 않은 경우에는 AlarmPresenter로 옮겼습니다.
- 그 결과, SoundManager는 '재생'과 '정지'라는 핵심 책임만 갖게 되었고,
유지보수성이 크게 좋아졌습니다.

6. GameStartManager

- 게임 로드 및 시작 과정은 메서드를 최대한 세분화했습니다.

```
public class GameStartManagerMono : MonoBehaviour
```

```
private void Awake()
{
    Initialize();

    PreLoadAudioDataAsync().Forget();

    DoNotDestroyUntilLoadBackgroundComplete();

    ShowGameLoadSceneBackgroundAsync().Forget();
}

private void Initialize()
{
    InitializeManagerTypesByReflection();

    CreateManagers();

    // note : 이 시점 이후에는 싱글톤이 null이 되지 않음이 보장된다.

    InitializeManagers();
}
```

- Manager의 Initialize,
Model의 Load,
UnityEngine.Object의 Awake() 실행 순서가 꼬이지 않도록 구현했습니다.

```
private void InitializeManagers()
{
    foreach (var manager in _managerList)
    {
        manager.PreInitialize();
    }

    InitializeModels();

    InjectModelsInManagers();

    foreach (var manager in _managerList)
    {
        manager.Initialize();
    }

    foreach (var manager in _managerList)
    {
        manager.BindEvent();
    }
}
```

6. GameStartManager

- 독립적으로 수행해도 되는 것은 비동기로 처리했습니다.

```
private async UniTaskVoid PreLoadAudioDataAsync()
{
    var alarmData = ScriptableObjectManager.Instance.GetScriptableObject<AlarmData>();
    if (alarmData == null)
    {
        throw new NullReferenceException("alarmData is null");
    }

    alarmData.Initialize();
    var sleepingAudioClipPathDictionary = alarmData.SleepingAudioClipPathDictionary;

    foreach (var element in sleepingAudioClipPathDictionary)
    {
        var key = element.Key;
        var relativePath = element.Value;

        var loadData = await Resources.LoadAsync<AudioClip>(relativePath);
        var memoryLoadedAudioClip = loadData as AudioClip;

        alarmData.SetSleepingAudioClipDictionary(key, memoryLoadedAudioClip);

        //Log
        Debug.Log($"{relativePath}의 음원 파일 {memoryLoadedAudioClip?.name}이 비동기로 로드 되었습니다");
    }

    ChangeScene();
}
```

6. GameStartManager

- 독립적으로 수행해도 되는 것은 비동기로 처리했습니다.

```
private async UniTaskVoid PreLoadAudioDataAsync()
{
    var alarmData = ScriptableObjectManager.Instance.GetScriptableObject<AlarmData>();
    if (alarmData == null)
    {
        throw new NullReferenceException("alarmData is null");
    }

    alarmData.Initialize();
    var sleepingAudioClipPathDictionary = alarmData.SleepingAudioClipPathDictionary;

    foreach (var element in sleepingAudioClipPathDictionary)
    {
        var key = element.Key;
        var relativePath = element.Value;

        var loadData = await Resources.LoadAsync<AudioClip>(relativePath);
        var memoryLoadedAudioClip = loadData as AudioClip;

        alarmData.SetSleepingAudioClipDictionary(key, memoryLoadedAudioClip);

        //Log
        Debug.Log($"{relativePath}의 음원 파일 {memoryLoadedAudioClip?.name}이 비동기로 로드 되었습니다");
    }

    ChangeScene();
}
```

7. 후기

- 간단한 기능들을 MVP 패턴으로 구현하며 책임 분리를 명확히 했습니다.
- 책임이 분리되니 버그의 개수와 버그의 수정 시간이 줄어들었습니다.
- Interface와 abstract를 통해 메서드의 바디를 구현하기 전에 이 클래스의 책임을 미리 설계할 수 있었습니다.
- 퇴사하기 두 달 전, 팀장님께서 ‘혼자서 간단한 슈팅게임을 만들 수 있니?’ 라고 물으셨을 때 저는 ‘못 할 것 같습니다.’ 라고 대답했습니다.
개인 프로젝트는 회사처럼 많은 콘텐츠가 연결되고, 서버와 협업하며, 타인의 코드를 수정하고 의도를 파악하는 연습을 제공해 주지는 않았습니다.

하지만 직장 생활 중 고민하던 질문들에 스스로 답해 보았고,
그때 해결하지 못했던 부분을 이번 프로젝트에서 보완하며 완성했습니다.

- 이제는 혼자서도 게임 개발을 할 수 있을 것 같다고 자신 있게 말할 수 있습니다.

감사합니다!